UNIT-I

Structures and Unions: Introduction, Nested Structures, Arrays of Structures, Structures and Functions, Self-Referential Structures, Unions, Enumerated Data Type - enum variables, Using Typedef keyword, Bit Fields.
Data Structures: Introduction to Data Structures, Types of Data Structures.

Structure:

In C, a structure is a user-defined data type that can be used to group items of possibly different types into a single type. The struct keyword is used to define a structure. The items in the structure are called its member and they can be of any valid data type.

Creation of a Structure:

A structure is defined using the **struct** keyword followed by the structure name and its members. It is also called a **structure template** or **structure prototype**, and no memory is allocated to the structure in the declaration.

```
struct structure_name {
        data_type1 member1;
        data_type2 member2;
        …
};
```

- **structure_name:** Name of the structure.
- **member1, member2, …:** Name of the members.
- **data_type1**, **data_type2**, …: Type of the members.

Ex:

```
struct A {
   int x;
};
```

Creating Structure Variable:

After structure definition, we have to create variable of that structure to use it. It is similar to the any other type of variable declaration.

We can create structure variable in two way—

- create a structure variable separately.
- create a structure variable along with the definition.

```
struct strcuture_name var;
  struct A {
     int x;
  };
  struct A a;
```

We can also declare structure variables with structure definition—

```
struct structure_name {
…
}var1, var2….;
```

Ex:

```
struct A{
        int x;
}a;
```

Basic Operations of Structure

1. Access Structure Members

To access or modify members of a structure, we use the **(.) dot operator**. This is applicable when we are using structure variables directly.

```
structure_variable_name . member1;
strcuture_variable_name . member2;
```

Ex:

```
struct student{
   char name[10];
   int rollno;
   int marks;
};
struct student s;
s.name
s.rollno
s.marks
```

In the case where we have a pointer to the structure, we can also use the **arrow operator** to access the members.

```
structure_ptr -> member1
structure_ptr -> member2
```

2. Initialize Structure Members

Structure members **cannot be** initialized with the declaration. For example, the following C program fails in the compilation.

```
struct structure_name {
data_type1 member1 = value1; // COMPILER ERROR: cannot initialize members here
data_type2 member2 = value2; // COMPILER ERROR: cannot initialize members here
…
};
```

The reason for the above error is simple. When a datatype is declared, no memory is allocated for it. Memory is allocated only when variables are created. So there is no space to store the value assigned.

We can initialize structure members in 4 ways which are as follows:

Default Initialization

By default, structure members are not automatically initialized to 0 or NULL. Uninitialized structure members will contain garbage values. However, when a structure variable is declared with an initializer, all members not explicitly initialized are zero-initialized.

struct structure_name = {0}; // Both x and y are initialized to 0

Initialization using Assignment Operator

struct structure_name str;
str.member1 = value1;
….

Note: We cannot initialize the arrays or strings using assignment operator after variable declaration.

Initialization using Initializer List

struct structure_name str = {value1, value2, value3 ….};

In this type of initialization, the values are assigned in sequential order as they are declared in the structure template.

Example:

```
#include <stdio.h>
// Defining a structure to represent a student
struct Student {
   char name[50];
   int age;
   float grade;
};
int main() {
   // Declaring and initializing a structure
   // variable
   struct Student s1 = {"Rahul",20, 18.5};
   // Designated Initializing another stucture
     struct Student s2 = {.age = 18, .name =  "Vikas", .grade = 22};
   // Accessing structure members
   printf("%s\t%d\t%.2f\n", s1.name, s1.age, s1.grade);
   printf("%s\t%d\t%.2f\n", s2.name, s2.age, s2.grade);
   return 0;
}
```

Output:

```
Rahul    20    18.50
Vikas    18    22.00
```

3. Copy Structure

Copying structure is simple as copying any other variables. For example, s1 is copied into s2 using assignment operator.

s2 = s1;

But this method only creates a shallow copy of s1 i.e. if the structure s1 have some dynamic resources allocated by malloc, and it contains pointer to that resource, then only the pointer will

be copied to s2. If the dynamic resource is also needed, then it has to be copied manually (deep copy).

Example:

```
#include <stdio.h>
#include <stdlib.h>
struct Student {
   int id;
    char grade;
};
int main() {
   struct Student s1 = {1, 'A'};
    // Create a copy of student s1
    struct Student s1c = s1;
   printf("Student 1 ID: %d\n", s1c.id);
   printf("Student 1 Grade: %c", s1c.grade);
    return 0;
}
```

Output:

```
Student 1 ID: 1
Student 1 Grade: A
```

Array of Structures in C:

An array of structures is simply an array where each element is a structure. It allows you to store several structures of the same type in a single array.

Array of Structure Declaration

Once you have already defined structure, the array of structure can be defined in a similar way as any other variable.

```
struct struct_name arr_name [size];
```

Need for Array of Structures

Suppose we have 50 employees, and we need to store the data of 50 employees. So for that, we need to define 50 variables of struct Employee type and store the data within that. However, declaring and handling the 50 variables is not an easy task. Let's imagine a bigger scenario, like 1000 employees.

So, if we declare the variable this way, it's not possible to handle this.

struct Employee emp1, emp2, emp3, ... emp1000;

For that, we can define an array whose data type will be struct Employee soo that will be easily manageable.

- Arrays of structures allow you to group related data together and handle multiple instances of that data more efficiently.

- It is particularly useful when you want to manage large datasets (such as a list of employees, students, or products) where each element has a consistent structure.

Basic Operations on Array of Structures

Following are the basic operations on array of structures:

Initialization

A structure can be initialized using initializer list and so can be the array. Therefore, we can initialize the array of structures using nested initializer list:

```
struct struct_name arr_name [size] = {
{element1_value1, element1_value2, ....},
{element2_value1, element2_value2, ....},
......
......
};
```

We can also skip the nested braces, but it is not recommended as we can easily lose the count of the element/member and may mess up the initialization.

```
struct struct_name arr_name [size]= {
element1_value1, element1_value2 ....,
element2_value1, element2_value2 .....
};
```

Access and Update Members

To access the members of a structure in an array, use the index of the array element followed by the dot (.) operator. The general syntax is:

```
arr_name[index].member_name
```

where,

- **arr_name**: The name of the array of structures.
- **index**: The index of the structure element in the array.
- **member_name**: The member within the structure you want to access.

Example of Array of Structure

The below code demonstrates the application of array of structure inside a C program:

Store Student Information

```
#include <stdio.h>
#include <string.h>
// Structure definition
struct Student {
   char name[50];
   int age;
   float marks;
};

int main() {
```

```
    // Declaration and initialization of an array of structures
    struct Student students[3] = {
        {"Nikhil", 20, 85.5},
        {"Shubham", 22, 90.0},
        {"Vivek", 25, 78.0}
    };
    // Traversing through the array of structures and displaying the data
    for (int i = 0; i < 3; i++) {
        printf("Student %d:\n", i+1);
        printf("Name: %s\n", students[i].name);
        printf("Age: %d\n", students[i].age);
        printf("Marks: %.2f\n\n", students[i].marks);
    }
    return 0;
}
```

Output:

```
Student 1:
Name: Nikhil
Age: 20
Marks: 85.50

Student 2:
Name: Shubham
Age: 22
Marks: 90.00

Student 3:
Name: Vivek
Age: 25
Marks: 78.00
```

Nested Structure in C:

A nested structure in C is a structure within structure. One structure can be declared inside another structure in the same way structure members are declared inside a structure.

The structure can be nested in the following different ways:

1. **By separate nested structure**
2. **By embedded nested structure**.

1. **By separate nested structure:** In this method, the two structures are created, but the dependent structure(Employee) should be used inside the main structure(Organisation) as a member.

```
// C program to implement
// the above approach
#include <stdio.h>
#include <string.h>
// Declaration of the
// dependent structure
struct Employee
{
int employee_id;
char name[20];
int salary;
};
// Declaration of the
// Outer structure
struct Organisation
{
char organisation_name[20];
char org_number[20];
// Dependent structure is used
// as a member inside the main
// structure for implementing
// nested structure
struct Employee emp;
};
// Driver code
int main()
{
// Structure variable
struct Organisation org;
// Print the size of organisation
// structure
printf("The size of structure organisation : %ld\n", sizeof(org));
org.emp.employee_id = 101;
strcpy(org.emp.name, "Robert");
org.emp.salary = 400000;
strcpy(org.organisation_name, "Aditya University");
strcpy(org.org_number, "ADTPPU");
// Printing the details
printf("Organisation Name : %s\n", org.organisation_name);
printf("Organisation Number : %s\n", org.org_number);
```

```
printf("Employee id : %d\n",  org.emp.employee_id);
printf("Employee name : %s\n", org.emp.name);
printf("Employee Salary : %d\n", org.emp.salary);
}
```

Output:

```
The size of structure organisation : 68
Organisation Name : Aditya University
Organisation Number : ADTPPU
Employee id : 101
Employee name : Robert
Employee Salary : 400000
```

2. **By Embedded nested structure:** Using this method, allows to declare structure inside a structure and it requires fewer lines of code.

   Whenever an embedded nested structure is created, the variable declaration is compulsory at the end of the inner structure, which acts as a member of the outer structure. It is compulsory that the **structure variable** is created at the end of the inner structure.

```
// C program to implement
// the above approach
#include <stdio.h>
#include <string.h>
// Declaration of the main
// structure
struct Organisation
{
char organisation_name[20];
char org_number[20];
// Declaration of the dependent
// structure
struct Employee
{
        int employee_id;
        char name[20];
        int salary;
// variable is created which acts
// as member to Organisation structure.
} emp;
};
// Driver code
int main()
{
```

```
struct Organisation org;
// Print the size of organisation structure
printf("The size of structure organisation : %ld\n", sizeof(org));
org.emp.employee_id = 101;
strcpy(org.emp.name, "Robert");
org.emp.salary = 400000;
strcpy(org.organisation_name,"Aditya Unverstiy");
strcpy(org.org_number, "ADTPPU");
// Printing the details
printf("Organisation Name : %s\n", org.organisation_name);
printf("Organisation Number : %s\n", org.org_number);
printf("Employee id : %d\n", org.emp.employee_id);
printf("Employee name : %s\n", org.emp.name);
printf("Employee Salary : %d\n", org.emp.salary);
}
```

Output:

```
The size of structure organisation : 68
Organisation Name : Aditya University
Organisation Number : ADTPPU
Employee id : 101
Employee name : Robert
Employee Salary : 400000
```

Structures and Functions:

When passing structures to or from functions in C, it is important to keep in mind that the entire structure will be copied. This can be expensive in terms of both time and memory, especially for large structures. The passing of structure to the function can be done in two ways:

- By passing all the elements to the function individually.
- By passing the entire structure to the function.

Example 1: Using Call By Value Method

```
//C program to pass structure as an argument to the
// functions using Call By Value Method
#include <stdio.h>
struct car {
        char name[30];
        int price;
};
void print_car_info(struct car c)
{
        printf("Name : %s", c.name);
        printf("\nPrice : %d\n", c.price);
}
int main()
```

Example 2: Using Call By Reference Method

```
// C program to pass structure as an argument to the
// functions using Call By Reference Method
#include <stdio.h>
struct student {
        char name[50];
        int roll;
        float marks;
};
void display(struct student* student_obj)
{
        printf("Name: %s\n", student_obj->name);
        printf("Roll: %d\n", student_obj->roll);
        printf("Marks: %f\n", student_obj->marks);
```

```
{
        struct car c = { "Tata", 1021 };
        print_car_info(c);
        return 0;
}
```

Output

Name : Tata
Price : 1021

```
}
int main()
{
        struct student st1 = { "Aman", 19, 8.5 };
        display(&st1);
        return 0;
}
```

Output

Name: Aman
Roll: 19
Marks: 8.500000

How to Return a Structure From functions?

We can return a structure from a function using the **return** Keyword. To return a structure from a function the return type should be a structure only.

```
// C program to return a structure from a function
#include <stdio.h>
struct student {
        char name[20];
        int age;
        float marks;
};
// function to return a structure
struct student get_student_data()
{
        struct student s;
        printf("Enter name: ");
        scanf("%s", s.name);
        printf("Enter age: ");
        scanf("%d", &s.age);
        printf("Enter marks: ");
        scanf("%f", &s.marks);
        return s;
}
int main()
{
        // structure variable s1 which has been assigned the
        // returned value of get_student_data
        struct student s1 = get_student_data();
        // displaying the information
        printf("Name: %s\n", s1.name);
        printf("Age: %d\n", s1.age);
        printf("Marks: %.1f\n", s1.marks);
```

```
    return 0;
}
```

Output:

```
Enter name: Krishna
Enter age: 21
Enter marks: 99
Name: Krishna
Age: 21
Marks: 99.0
```

Unions in C:

In C, union is a user-defined data type that can contain elements of the different data types just like structure. But unlike structures, all the members in the C union are stored in the same memory location. Due to this, only one member can store data at the given point in time.

C Union Declaration

In this part, we only declare the template of the union, i.e., we only declare the members' names and data types along with the name of the union. No memory is allocated to the union in the declaration.

```
union name {
type1 member1;
type2 member2;
.
.
};
```

Create a Union Variable

We need to define a variable of the union type to start using union members. There are two methods using which we can define a union variable:

Creating Union Variable with Declaration

```
union name{
type member1;
type member2;
…
} var1, var2, …;
```

Creating Union Variable after Declaration

```
union name var1, var2, var3…;
```

where ***name*** is the name of an already declared union.

Access Union Members

We can access the members of a union by using the (.) dot operator just like structures.

```
var1.member1;
```

where ***var1*** is the **union variable** and ***member1*** is the **member of the union**.

Initialize Union

The initialization of a union is the initialization of its members by simply assigning the value to it.

```
var1.member1 = val;
```

One important thing to note here is that **only one member can contain some value at a given instance of time.**

Size of Union

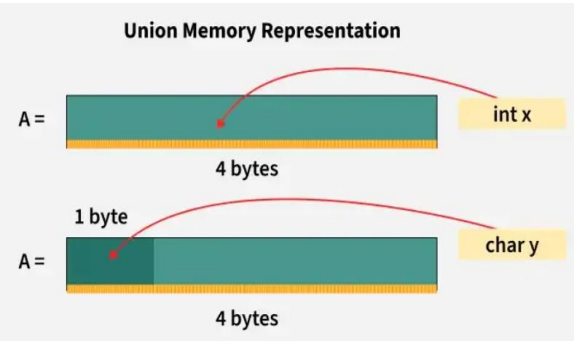
The size of the union will always be equal to the **size of the largest member** of the union. All the less-sized elements can store the data in the same space without any overflow.

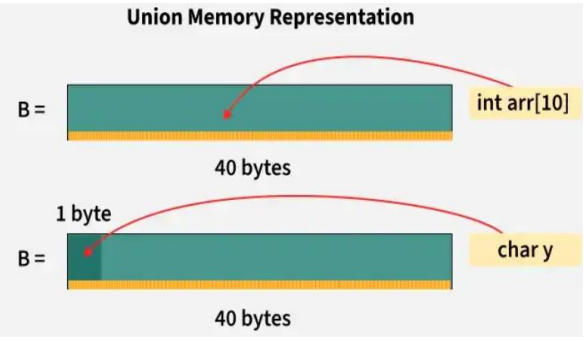
```
#include <stdio.h>
// Declaring multiple unions
union A{
   int x;
   char y;
};
union B{
   int arr[10];
   char y;
};
int main() {
   // Finding size using sizeof() operator
   printf("Sizeof A: %ld\n", sizeof(union A));
   printf("Sizeof B: %ld\n", sizeof(union B));
   return 0;
}
```

Output

```
Sizeof A: 4
Sizeof B: 40
```

The above unions memory can be visualized as shown:



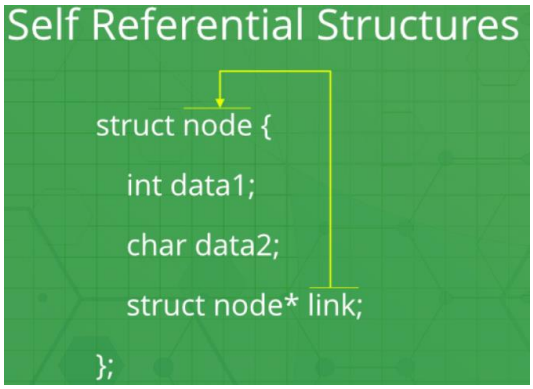


Difference Between Structure and Union in C

Parameter	Structure	Union
Definition	A structure is a user-defined data type that groups different data types into a single entity.	A union is a user-defined data type that allows storing different data types at the same memory location.
Keyword	The keyword **struct** is used to define a structure	The keyword **union** is used to define a union
Size	The size is the sum of the sizes of all members, with padding if necessary.	The size is equal to the size of the largest member, with possible padding.
Memory Allocation	Each member within a structure is allocated unique storage area of location.	Memory allocated is shared by individual members of union.
Data Overlap	No data overlap as members are independent.	Full data overlap as members shares the same memory.
Accessing Members	Individual member can be accessed at a time.	Only one member can be accessed at a time.

Self Referential Structures:

Self Referential structures are those structures that have one or more pointers which point to the same type of structure, as their member.

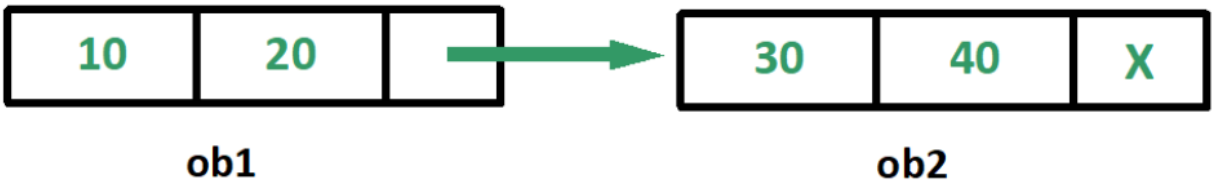


In other words, structures pointing to the same type of structures are self-referential in nature

Types of Self Referential Structures

1. Self Referential Structure with Single Link
2. Self Referential Structure with Multiple Links

Self Referential Structure with Single Link: These structures can have only one self-pointer as their member. The following example will show us how to connect the objects of a self-referential structure with the single link and access the corresponding data members. The connection formed is shown in the following figure.



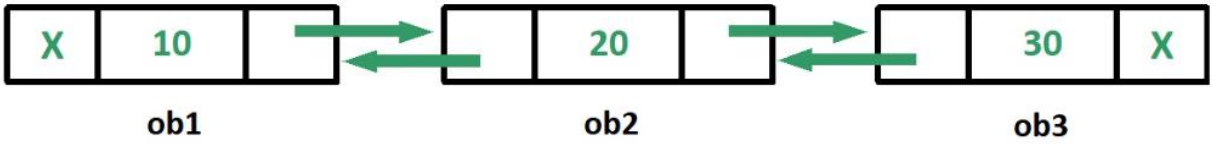
Program:

```
#include <stdio.h>
struct node {
   int data1;
   char data2;
   struct node* link;
};
int main(){
   struct node ob1; // Node1
   // Initialization
   ob1.link = NULL;
   ob1.data1 = 10;
   ob1.data2 = 20;
   struct node ob2; // Node2
   // Initialization
   ob2.link = NULL;
   ob2.data1 = 30;
   ob2.data2 = 40;
   // Linking ob1 and ob2
   ob1.link = &ob2;
   // Accessing data members of  ob2 using ob1
   printf("%d", ob1.link->data1);
   printf("\n%d", ob1.link->data2);
   return 0;
}
```

Output

30
40

Self Referential Structure with Multiple Links: Self referential structures with multiple links can have more than one self-pointers. Many complicated data structures can be easily constructed using these structures. Such structures can easily connect to more than one nodes at a time. The following example shows one such structure with more than one links. The connections made in the above example can be understood using the following figure.



Program:

```
#include <stdio.h>
struct node {
   int data;
   struct node* prev_link;
   struct node* next_link;
};
int main()
{
   struct node ob1; // Node1
   // Initialization
   ob1.prev_link = NULL;
   ob1.next_link = NULL;
   ob1.data = 10;
   struct node ob2; // Node2
   // Initialization
   ob2.prev_link = NULL;
   ob2.next_link = NULL;
   ob2.data = 20;
   struct node ob3; // Node3
   // Initialization
   ob3.prev_link = NULL;
   ob3.next_link = NULL;
   ob3.data = 30;
   // Forward links
   ob1.next_link = &ob2;
   ob2.next_link = &ob3;
   // Backward links
   ob2.prev_link = &ob1;
   ob3.prev_link = &ob2;
   // Accessing  data of ob1, ob2 and ob3 by ob1
   printf("%d\t", ob1.data);
```

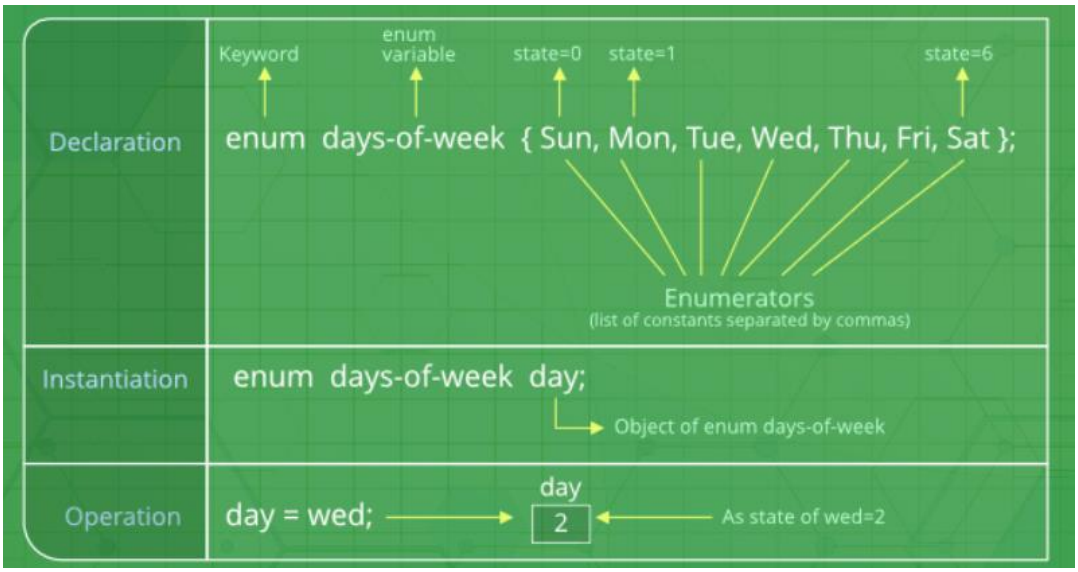
```
    printf("%d\t", ob1.next_link->data);
    printf("%d\n", ob1.next_link->next_link->data);
    // Accessing data of ob1, ob2 and ob3 by ob2
    printf("%d\t", ob2.prev_link->data);
    printf("%d\t", ob2.data);
    printf("%d\n", ob2.next_link->data);
    // Accessing data of ob1, ob2 and ob3 by ob3
    printf("%d\t", ob3.prev_link->prev_link->data);
    printf("%d\t", ob3.prev_link->data);
    printf("%d", ob3.data);
    return 0;
}
```

Output

```
10    20    30
10    20    30
10    20    30
```

Enumeration (or enum) in C:

Enumeration (or enum) is a user defined data type in C. It is mainly used to assign names to integral constants, the names make a program easy to read and maintain.



enum State {Working = 1, Failed = 0};

The keyword **'enum'** is used to declare new enumeration types in C. Enums in C allow you to assign names to integral constants, making code more readable.

Following is an example of enum declaration.

enum flag{constant1, constant2, constant3, ... };

The name of enumeration is "flag" and the constant are the values of the flag. By default, the values of the constants are as follows:
constant1 = 0, constant2 = 1, constant3 = 2 and so on.

- Variables of type enum can also be defined. They can be defined in two ways:

```
// In both of the below cases, "day" is
// defined as the variable of type week.
enum week{Mon, Tue, Wed};
enum week day;
// Or
enum week{Mon, Tue, Wed}day;
```

An example program to demonstrate working of enum in C

```
#include<stdio.h>
enum week{Mon, Tue, Wed, Thur, Fri, Sat, Sun};
int main()
{
   enum week day;
   day = Wed;
   printf("%d",day);
   return 0;
}
```

Output:

2

In the above example, we declared “day” as the variable and the value of “Wed” is allocated to day, which is 2. So as a result, 2 is printed.

Another example program to demonstrate working of enum in C

```
#include<stdio.h>
enum year{Jan, Feb, Mar, Apr, May, Jun, Jul, Aug, Sep, Oct, Nov, Dec};
int main()
{
  int i;
  for (i=Jan; i<=Dec; i++)
    printf("%d ", i);
  return 0;
}
```

Output:

0 1 2 3 4 5 6 7 8 9 10 11

typedef keyword:

The **typedef** is a keyword that is used to provide existing data types with a new name. The C typedef keyword is used to redefine the name of already existing data types. When names of datatypes become difficult to use in programs, typedef is used with user-defined datatypes, which behave similarly to defining an alias for commands.

Syntax of typedef

typedef existing_type new_type;

where,

- **existing_type:** The type that we want to alias (e.g., int, float, struct, etc.).
- **new_type:** The new alias or name for the existing type.

After this declaration, we can use the ***alias_name*** as if it were the real ***existing_name*** in out C program.

Examples of typedef in C

The below examples illustrate the use of typedef for different purposes in our C program:

```
#include <stdio.h>
// Defining an alias using typedef
typedef long long ll;
int main() {
   // Using typedef alias name to declare variable
   ll a = 20;
   printf("%lld", a);
   return 0;
}
```

Output

20

Explanation: In this code, typedef is used to define an alias **ll** for the **long long data type**. The variable **a** is declared using **ll** as **a** shorthand, and its value is printed using printf. This makes the code more readable and concise by using the alias instead of the full type of name.

Define an Alias for a Structure

```
#include <stdio.h>
#include <string.h>
// Using typedef to define an alias for structure
typedef struct Students {
   char name[50];
   char branch[50];
   int ID_no;
} stu;
int main() {
    // Using alias to define structure
   stu s;
   strcpy(s.name, "Aditya");
```

```
    strcpy(s.branch, "CSE");
    s.ID_no = 108;
    printf("%s\n", s.name);
    printf("%s\n", s.branch);
    printf("%d", s.ID_no);
    return 0;
}
```

Output

```
Aditya
CSE
108
```

Explanation: In this code, typedef is used to define an alias **stu** for the structure **Students**. The alias simplifies declaring variables of this structure type, such as **st**. The program then initializes and prints the values of the structure members name, branch, and ID_no. This approach enhances code readability by using the alias instead of the full **struct students** declaration.

Bit Fields in C

In C, we can specify the size (in bits) of the structure and union members. The idea of bit-field is to use memory efficiently when we know that the value of a field or group of fields will never exceed a limit or is within a small range. C Bit fields are used when the storage of our program is limited.

Need of Bit Fields in C

- Reduces memory consumption.
- To make our program more efficient and flexible.
- Easy to Implement.

Declaration of C Bit Fields

Bit-fields are variables that are defined using a predefined width or size. Format and the declaration of the bit-fields in C are shown below:

Syntax of C Bit Fields

```
struct structure_name
{
   data_type member_name : width_of_bit-field;
};
```

where,

- **data_type:** It is an integer type that determines the bit-field value which is to be interpreted. The type may be int, signed int, or unsigned int.
- **member_name:** The member name is the name of the bit field.
- **width_of_bit-field:** The number of bits in the bit-field. The width must be less than or equal to the bit width of the specified type.

Example of C Bit Fields

In this example, we compare the size difference between the structure that does not specify bit fields and the structure that has specified bit fields.

Structure Without Bit Fields

Consider the following declaration of date without the use of bit fields.

C Program to illustrate the structure without bit field

```
#include <stdio.h>
// A simple representation of the date
struct date {
   unsigned int d;
   unsigned int m;
   unsigned int y;
};
int main()
{
   // printing size of structure
   printf("Size of date is %lu bytes\n",sizeof(struct date));
   struct date dt = { 31, 12, 2014 };
   printf("Date is %d/%d/%d", dt.d, dt.m, dt.y);
}
```

Output

```
Size of date is 12 bytes
Date is 31/12/2014
```

The above representation of 'date' takes 12 bytes on a compiler whereas an unsigned int takes 4 bytes. Since we know that the value of d is always from 1 to 31, and the value of m is from 1 to 12, we can optimize the space using bit fields.

Structure with Bit Field:

```
// C program to demonstrate use of Bit-fields
#include <stdio.h>
// Space optimized representation of the date
struct date {
   // d has value between 0 and 31, so 5 bits are sufficient
   int d : 5;
   // m has value between 0 and 15, so 4 bits
   // are sufficient
   int m : 4;
   int y;
};
int main()
{
   printf("Size of date is %lu bytes\n", sizeof(struct date));
```

```
    struct date dt = { 31, 12, 2014 };
    printf("Date is %d/%d/%d", dt.d, dt.m, dt.y);
    return 0;
}
```

Output

Size of date is 8 bytes
Date is -1/-4/2014

Explanation

The output comes out to be negative. What happened behind is that the value 31 was stored in 5 bit signed integer which is equal to 11111. The MSB is a 1, so it's a negative number and you need to calculate the 2's complement of the binary number to get its actual value which is what is done internally.

By calculating 2's complement you will arrive at the value 00001 which is equivalent to the decimal number 1 and since it was a negative number you get a -1. A similar thing happens to 12 in which case you get a 4-bit representation as 1100 and on calculating 2's complement you get the value of -4.

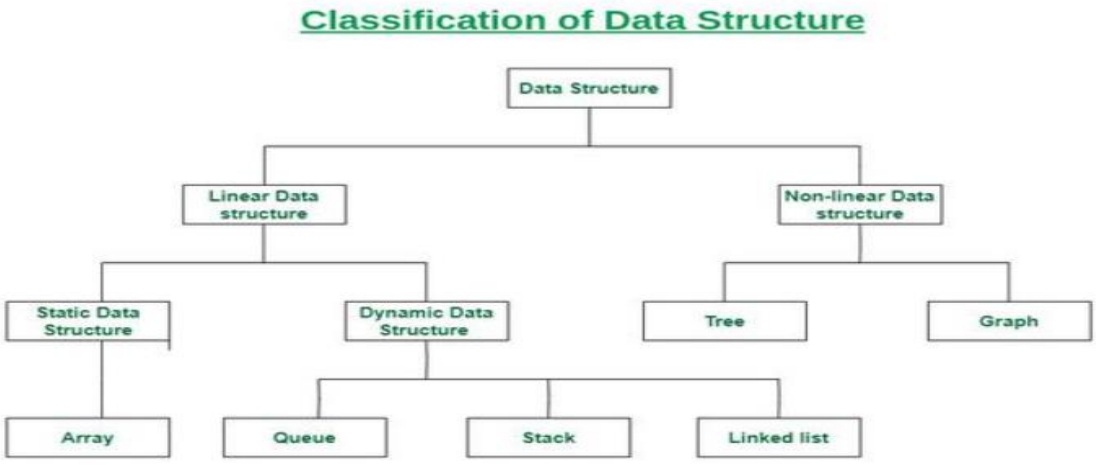
Data Structure:

A **data structure** is a storage that is used to store and organize data. It is a way of arranging data on a computer so that it can be accessed and updated efficiently.

A data structure organizes, processes, retrieves, and stores data, making it essential for nearly every program or software system. To help you master them, we've compiled a comprehensive guide covering types, classifications, and applications of data structures. This article simplifies everything, helping you choose the right one in minutes.

Classification of Data Structure

Data structure has many different uses in our daily life. There are many different data structures that are used to solve different mathematical and logical problems. By using data structure, one can organize and process a very large amount of data in a relatively short period. Let's look at different data structures that are used in different situations.



Classification of Data Structure

- **Linear data structure:** Data structure in which data elements are arranged sequentially or linearly, where each element is attached to its previous and next adjacent elements, is called a linear data structure.
  Examples: array, stack, queue, linked list, etc.
  - **Static data structure:** Static data structure has a fixed memory size. It is easier to access the elements in a static data structure.
    Example: array data structure.
  - **Dynamic data structure:** In the dynamic data structure, the size is not fixed. It can be randomly updated during the runtime which may be considered efficient concerning the memory (space) complexity of the code.
    Examples: stack and queue data structures.
- **Non-linear data structure:** Data structures where data elements are not placed sequentially or linearly are called non-linear data structures. In a non-linear data structure, we can't traverse all the elements in a single run.
  Examples: tree and graph data structures.